APMA 4302 - Homework 3

Jin Hu

March 2026

1. Use finite differences to discretize the nonlinear PDE and write down the resulting nonlin- ear system of equations in the form F (u) = 0. Include auxiliary equations for non-homogeneous Dirichlet boundary conditions.

solution:

For

$$-\nabla^2 u + \gamma u^p = f \quad \text{in } \Omega = (0,1) \times (0,1),$$

with boundary conditions

$$u(x,y) = u_{\text{exact}}(x,y) \quad \text{on } \partial\Omega,$$

where

$$u_{\text{exact}}(x,y) = A \exp\left(-\frac{(x-x_0)^2 + (y-y_0)^2}{\sigma^2} \right).$$

Let the unit square be discretized by a uniform grid with spacing h, with grid points

$$x_i = ih, \qquad y_j = jh,$$

for $i, j = 0, 1, \ldots, N$, where $h = 1/N$. Let $u_{i,j} \approx u(x_i, y_j)$.

For interior grid points $1 \le i, j \le N-1$, we approximate the Laplacian with the standard five-point stencil:

$$-\nabla^2 u(x_i, y_j) \approx \frac{4u_{i,j} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1}}{h^2}.$$

Therefore, for each interior point, the discrete nonlinear equation is

$$F_{i,j}(u) = \frac{4u_{i,j} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1}}{h^2} + \gamma (u_{i,j})^p - f_{i,j} = 0,$$

where $f_{i,j} = f(x_i, y_j)$.

For boundary grid points, the Dirichlet condition is imposed directly. Thus, for (i,j) on the boundary,

$$F_{i,j}(u) = u_{i,j} - u_{\text{exact}}(x_i, y_j) = 0.$$

Hence the full nonlinear system can be written as

$$F_{i,j}(u) = \begin{cases} \dfrac{4u_{i,j} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1}}{h^2} + \gamma (u_{i,j})^p - f_{i,j}, & \text{for } 1 \le i, j \le N-1, \\ u_{i,j} - u_{\text{exact}}(x_i, y_j), & \text{for } (i,j) \in \partial\Omega_h. \end{cases}$$

This is the nonlinear algebraic system to be solved.

2. Provide equations for elements of the Jacobian matrix $J_{i,j}(\mathbf{u}) = \frac{\partial F_i}{\partial u_j}$.

solution:

Given

$$J_{(i,j),(k,\ell)}(u) = \frac{\partial F_{i,j}}{\partial u_{k,\ell}}.$$

For an interior grid point (i, j), the nonlinear residual is

$$F_{i,j}(u) = \frac{4u_{i,j} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1}}{h^2} + \gamma(u_{i,j})^p - f_{i,j}.$$

Therefore, the nonzero Jacobian entries in the row corresponding to (i, j) are

$$\frac{\partial F_{i,j}}{\partial u_{i,j}} = \frac{4}{h^2} + \gamma p(u_{i,j})^{p-1},$$

and

$$\frac{\partial F_{i,j}}{\partial u_{i-1,j}} = \frac{\partial F_{i,j}}{\partial u_{i+1,j}} = \frac{\partial F_{i,j}}{\partial u_{i,j-1}} = \frac{\partial F_{i,j}}{\partial u_{i,j+1}} = -\frac{1}{h^2}.$$

All other derivatives in that row are zero.

For a boundary grid point (i, j), we impose the Dirichlet condition directly:

$$F_{i,j}(u) = u_{i,j} - u_{\text{exact}}(x_i, y_j).$$

Hence, for a boundary row,

$$\frac{\partial F_{i,j}}{\partial u_{i,j}} = 1, \qquad \frac{\partial F_{i,j}}{\partial u_{k,\ell}} = 0 \quad \text{for } (k,\ell) \neq (i,j).$$

Thus, we have

$$J_{(i,j),(k,\ell)}(u) = \begin{cases} \dfrac{4}{h^2} + \gamma p(u_{i,j})^{p-1}, & \text{if } (k,\ell) = (i,j) \text{ and } (i,j) \text{ is interior,} \\ -\dfrac{1}{h^2}, & \text{if } (k,\ell) \in \{(i-1,j),(i+1,j),(i,j-1),(i,j+1)\} \\ & \text{and } (i,j) \text{ is interior,} \\ 1, & \text{if } (k,\ell) = (i,j) \text{ and } (i,j) \text{ is on the boundary,} \\ 0, & \text{otherwise.} \end{cases}$$

This Jacobian is used in Newton's method for solving the nonlinear system.

3. Modify the code reaction.c in the p4pdes repository to solve the nonlinear problem using Newton's method with the PETSc SNES objects. Your code should have the following prop- erties.

solution:

Discretization

We discretize the Laplacian using a standard second-order finite difference scheme on a uniform grid. For an interior grid point (i, j), the discrete residual is

$$F_{i,j}(u) = \frac{2u_{i,j} - u_{i-1,j} - u_{i+1,j}}{h_x^2} + \frac{2u_{i,j} - u_{i,j-1} - u_{i,j+1}}{h_y^2} + \gamma u_{i,j}^p - f_{i,j}.$$

For boundary nodes, Dirichlet conditions are imposed directly:

$$F_{i,j}(u) = u_{i,j} - u_{\text{exact}}(x_i, y_j).$$

In the numerical implementation, the residual is multiplied by a factor $h_x h_y$ to avoid division by small grid spacing and improve computational efficiency. This scaling does not change the solution since the nonlinear system $F(u) = 0$ is equivalent to $cF(u) = 0$ for any nonzero constant c.

Jacobian

The Jacobian matrix is constructed analytically. For interior points, the diagonal entry is

$$\frac{\partial F_{i,j}}{\partial u_{i,j}} = \frac{2}{h_x^2} + \frac{2}{h_y^2} + \gamma p\, u_{i,j}^{p-1}.$$

The off-diagonal entries correspond to the standard five-point stencil:

$$-\frac{1}{h_x^2}, \qquad -\frac{1}{h_y^2}.$$

For boundary points, the Jacobian is simply

$$\frac{\partial F_{i,j}}{\partial u_{i,j}} = 1.$$

Numerical Implementation

The problem is solved using PETSc with a DMDA structured grid and the SNES nonlinear solver. The residual and Jacobian are implemented using local stencil operations.

The code supports the following command-line options:

- `-rct_p`: exponent p
- `-rct_gamma`: reaction coefficient γ

- `-rct_linear_f`: use the linear manufactured right-hand side

The solution is written in VTK format for visualization in ParaView. The full implementation is provided in the submitted file `reaction2d.c`.

4. Run your code with $\gamma = 0$ for a 65×65 grid and show that the linear and nonlinear solvers produce the same solution within error. Find an appropriate set of linear solver options and stopping conditions so that the nonlinear solver converges to a residual norm

$$\|F(\mathbf{u})\|_2 < 10^{-10}$$

in one Newton iteration.

Report the number of linear iterations taken at each Newton step and the final relative error

$$\frac{\|\mathbf{u} - \mathbf{u}_{\text{exact}}\|_2}{\|\mathbf{u}_{\text{exact}}\|_2}.$$

Hint: some useful command line options include `-ksp_monitor`, `-snes_monitor`, `-ksp_rtol` and `-ksp_atol`. Feel free to experiment with different linear solvers and preconditioners (including sparse-direct solvers), but your code should run in parallel. Save your preferred options in a file `options_file_gamma0` and include it with your submission.

solution:

To verify the implementation in the linear limit, I ran the nonlinear solver with

$$\gamma = 0$$

on a 65×65 grid using `-da_refine 3` together with `-rct_linear_f`. In this case, the nonlinear problem reduces to the linear Poisson problem.

I used the following solver options, stored in `options_file_gamma0`:

- `-da_grid_x 9`
- `-da_grid_y 9`
- `-da_refine 3`
- `-snes_monitor`
- `-ksp_monitor`
- `-snes_atol 1e-10`
- `-snes_rtol 0`
- `-snes_stol 0`

- `-ksp_type preonly`
- `-pc_type lu`

With these settings, the nonlinear solver converged in one Newton iteration. The initial residual norm was

$$\|F(u^{(0)})\|_2 = 1.771465989748,$$

and after one Newton step it was reduced to

$$\|F(u^{(1)})\|_2 = 2.015822008010 \times 10^{-14},$$

which is well below the required tolerance 10^{-10}.

The linear solve inside this Newton step also converged rapidly. The KSP residual norm decreased from

$$1.771465989748$$

to

$$1.968933202050 \times 10^{-14}.$$

Thus, the number of linear iterations taken in this Newton step was 1.

The final relative error was

$$\frac{\|u - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2} = 0.000524515.$$

Since the nonlinear solver converges in a single Newton iteration to machine precision when $\gamma = 0$, this confirms that the problem reduces to a linear system in this case and that the SNES-based nonlinear solver reproduces the linear solution up to solver tolerance. The computation was run with

```
mpirun -n 1 ./reaction2d -rct_gamma 0 -rct_p 3 -rct_linear_f \
-options_file options_file_gamma0
```

5. Show that you get the same solution with the finite-difference Jacobian and report any differences in convergence behavior or final error. Include the command line options you used to run with the finite-difference Jacobian in a file `options_file_fd` and include it with your submission. Compare the run times for the user-provided Jacobian and the finite-difference Jacobian and report any differences. Hint: use `-log_view` to get detailed timing information.

solution:

To verify the correctness of the Jacobian implementation, I solved the nonlinear problem with $\gamma = 100$ and $p = 3$ on a 65×65 grid using both the user-provided analytic Jacobian and PETSc's finite-difference Jacobian option `-snes_fd_color`.

Using the analytic Jacobian, the nonlinear solver converged in 6 Newton iterations, with final residual norm

$$\|F(u)\|_2 \approx 1.04 \times 10^{-10},$$

and relative error

$$\frac{\|u - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2} = 0.000202508.$$

The total runtime reported by `-log_view` was approximately 0.2166 seconds.

Using the finite-difference Jacobian, the solver converged to the same numerical solution. The final residual norm was again approximately

$$\|F(u)\|_2 \approx 1.04 \times 10^{-10},$$

and the final relative error was

$$\frac{\|u - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2} = 0.000202508.$$

The total runtime reported by `-log_view` was approximately 0.2036 seconds.

Therefore, the finite-difference Jacobian and the user-provided Jacobian produced the same solution within numerical tolerance, confirming that the analytic Jacobian implementation is correct. In this run, the measured runtimes were very similar, with the finite-difference Jacobian slightly faster, although this small difference is likely affected by the fact that PETSc was compiled in debugging mode.

6. Now set $\gamma = 100$ and $p = 3$ and run the nonlinear solver with the same stopping conditions as before. Report the number of Newton iterations taken, the number of linear iterations taken at each Newton step, and the final relative error

$$\frac{\|u - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2}.$$

Plot the convergence history of the nonlinear residual norm $\|F(u)\|_2$.

solution:

I solved the nonlinear problem with

$$\gamma = 100, \qquad p = 3$$

on a 65×65 grid using `-da_refine 3`, together with `-snes_monitor` and `-ksp_monitor`.

The nonlinear solver converged in 6 Newton iterations. The nonlinear residual norms decreased from

$$\|F(u)\|_2 = 1.838084130533$$

to

$$\|F(u)\|_2 \approx 1.0 \times 10^{-10}.$$

The final relative error was

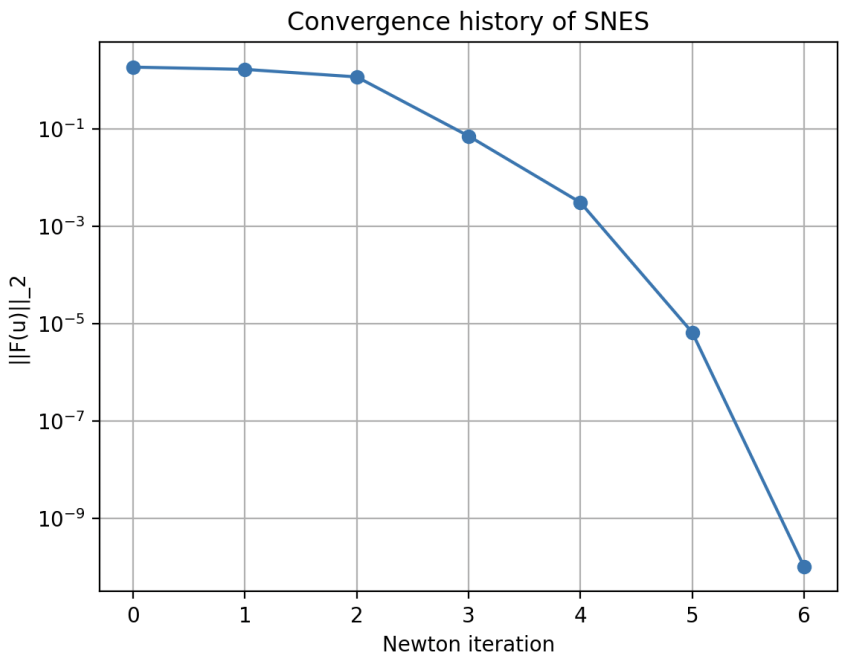
$$\frac{\|u - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2} = 0.000202508.$$

The number of linear (KSP) iterations required at each Newton step was as follows:

- Newton step 1: 46 linear iterations
- Newton step 2: 42 linear iterations
- Newton step 3: 37 linear iterations
- Newton step 4: 34 linear iterations
- Newton step 5: 33 linear iterations
- Newton step 6: 34 linear iterations

Overall, Newton's method converges robustly for the nonlinear reaction–diffusion problem. The first Newton step requires the most expensive linear solve, and the later steps are generally cheaper as the solution approaches convergence.

The convergence history of the nonlinear residual is shown here:


Convergence history of SNES
||F(u)||_2
10^{-1}
10^{-3}
10^{-5}
10^{-7}
10^{-9}
0
1
2
3
4
5
6
Newton iteration

7. rerun the problem with `-rct_linear_f` and visualize the solution for $\gamma = 100$ and $p = 3$ in ParaView and include a screenshot of the solution in your submission. Describe how the nonlinearity has affected the solution compared to the linear case $\gamma = 0$.

solution:

The SNES residual decreases rapidly over the Newton iterations, reaching

$$\|F(u)\|_2 \approx 5.2 \times 10^{-13},$$

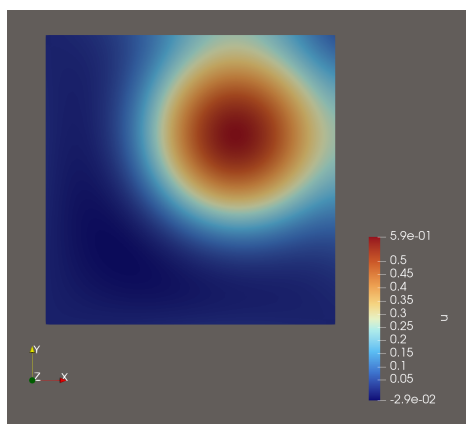
indicating good convergence of the nonlinear solver.

However, the relative error is large:

$$\frac{\|u - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2} \approx 0.41.$$

This is expected because the forcing term does not include the nonlinear contribution, so the exact solution is no longer a solution of the modified equation.

This Figure shows the computed solution u.



Compared to the linear case ($\gamma = 0$), the solution is noticeably modified by the nonlinear reaction term. In particular, the solution becomes more localized and the peak structure is altered, indicating that the nonlinear term significantly changes the shape and magnitude of the solution.

8. Performance and scaling make a table (and/or plot) comparing:

- run times (use `-log_view`)
- number of Newton iterations
- relative norm of the error

$$\frac{\|u - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2}$$

for the non-linear problem for `-da_refine 2, 3, 4, 5, 6` (i.e., grid sizes of 33×33 to 513×513) and $1, 2$, and 4 processors. I may actually provide a script for this part. Comment on the scaling of the code with respect to both problem size and number of processors.

solution:

I solved the nonlinear problem with

$$\gamma = 100, \qquad p = 3$$

for several grid sizes and processor counts. I used

$$-da_refine = 2, 3, 4, 5, 6,$$

corresponding to grid sizes 33×33, 65×65, 129×129, 257×257, and 513×513, respectively. For each case, I recorded the runtime from `-log_view`, the number of Newton iterations, and the relative error

$$\frac{\|u - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2}.$$

`da_refine`	Grid size	Processes	Runtime (s)	Newton iters	Relative error
2	33×33	1	0.1389	7	0.000809754
3	65×65	1	0.2067	6	0.000202508
4	129×129	1	0.6961	5	0.0000506486
5	257×257	1	6.357	7	0.000012663
6	513×513	1	66.95	7	0.00000316383
2	33×33	2	0.1432	7	0.000809754
3	65×65	2	0.2219	6	0.000202508
4	129×129	2	0.5713	5	0.0000506486
5	257×257	2	4.243	7	0.0000126627
6	513×513	2	43.07	7	0.00000316382
2	33×33	4	0.1734	7	0.000809754
3	65×65	4	0.2198	6	0.000202508
4	129×129	4	0.4370	5	0.0000506486
5	257×257	4	2.928	7	0.0000126629
6	513×513	4	25.67	7	0.0000031638

As the grid is refined, the relative error decreases consistently, indicating convergence of the numerical method. The error is nearly identical across different numbers of processes, as expected, since parallelization does not change the underlying discretization or solution.

The number of Newton iterations remains relatively stable (between 5 and 7) for all grid sizes and process counts, demonstrating robustness of the nonlinear solver.

The runtime increases significantly as the grid is refined due to the larger number of unknowns. When increasing the number of processes from 1 to 2 and 4, the runtime decreases for larger problem sizes (e.g., 257×257 and 513×513), showing effective parallel speedup. However, for small grids (e.g., 33×33), the runtime slightly increases with more processes due to communication overhead dominating computation.

Overall, the results demonstrate good scalability for larger problems, while parallel efficiency is limited for small problem sizes. This behavior is consistent with the expectation that parallel computing becomes more beneficial as the problem size increases.